

sat-mb-fidelity___fidelity-flat-f-k4

An AgentMPI run analysis

Abstract

This is the manager-summarises-everything cell of the saturation attempt, and because its root folds in one rank at a time it is the run that measures the campaign’s central mechanism directly. Eight ranks contribute twelve items each; rank 0 performs all seven merges in sequence; retention is 1.000 with no fabricated identifiers. Recovering the seven accumulators from the fabric gives item counts of 24, 36, 48, 60, 72, 84 and 96 — every rank fully absorbed — against output sizes of 354, 425, 373, 454, 535, 617 and 698 tokens. Two things follow. The operator re-encodes as pressure rises, from 39.3 bytes per item at the first merge to 14.3 at the last, converging on a packed table ([F-0-0]100n [F-0-1]101d ...) whose marginal cost is a flat 81 tokens per additional rank, or 6.8 tokens per item. And having re-encoded as far as it can, it crosses its 450-token budget at the fourth merge and overruns monotonically to 698 tokens, 55% over, without a single rejection — because the contract stored in this run’s fabric records `max_tokens:` `null` and an empty semantics string, so the budget existed only as a sentence in the prompt. That makes this run half of the most informative pair in my set. Its twin `inc-mb-fidelity-inc___fidelity-flat-f-k4-inc` is the same algorithm at the same p with the same nominal budget, but enforced and with incompressible content: it saturates at the same point in the fold, cannot overrun, and instead returns the identical accumulator four times while ranks 4 through 7 are silently erased. One knob — whether the runtime checks the number — separates retention 1.000 from retention 0.385. Note also that `flat` is not a one-shot p -way fold: `algorithms.py` applies the binary operator $p - 1$ times at the root, so this cell has `fold_depth:` 7 with rounds: 1.

1 What this run is

property	value
run	sat-mb-fidelity___fidelity-flat-f-k4
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	broker $\times$ 8, worker $\times$ 27
events	96
wall time	197.62s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.82×	total busy time over wall time
parallel efficiency	10.2%	achieved parallelism per rank
idle fraction	89.8%	rank-seconds paid for and unused
peak ranks busy	1	of 8 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	8.00×	slowest rank’s busy time over the mean
coordination share	12.5%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	27	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	27.51 / 31.91 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	2,709	0 deferred under rendezvous
tokens in / out	6,353 / 3,456	prompt and completion
cost	\$0.0709	0.0205 \$/kTok out

3 What the analysis notices

- **[critical]** At least one rank waited 6s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- **[warning]** Concurrency never exceeded one rank despite a world size of 8: this ran serially.
- **[warning]** The cost model’s α and β here are a median fallback, not a fit: the slope was negative ($\beta = -0.0141$ s/token), meaning latency fell as output grew — so something outside the model dominated the measurement, typically queueing, whose wait enters the latency but not the token count, on a fit with $R^2 = 0.419$. Any latency prediction from this run’s calibration is a median, not a model.
- **[note]** Load imbalance is 8.00×: the slowest rank was busy far longer than the mean.
- **[ok]** 27 rank reattachment(s) occurred, up to incarnation 10: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- **[ok]** All 1 checkable collective(s) also matched the predicted round count.

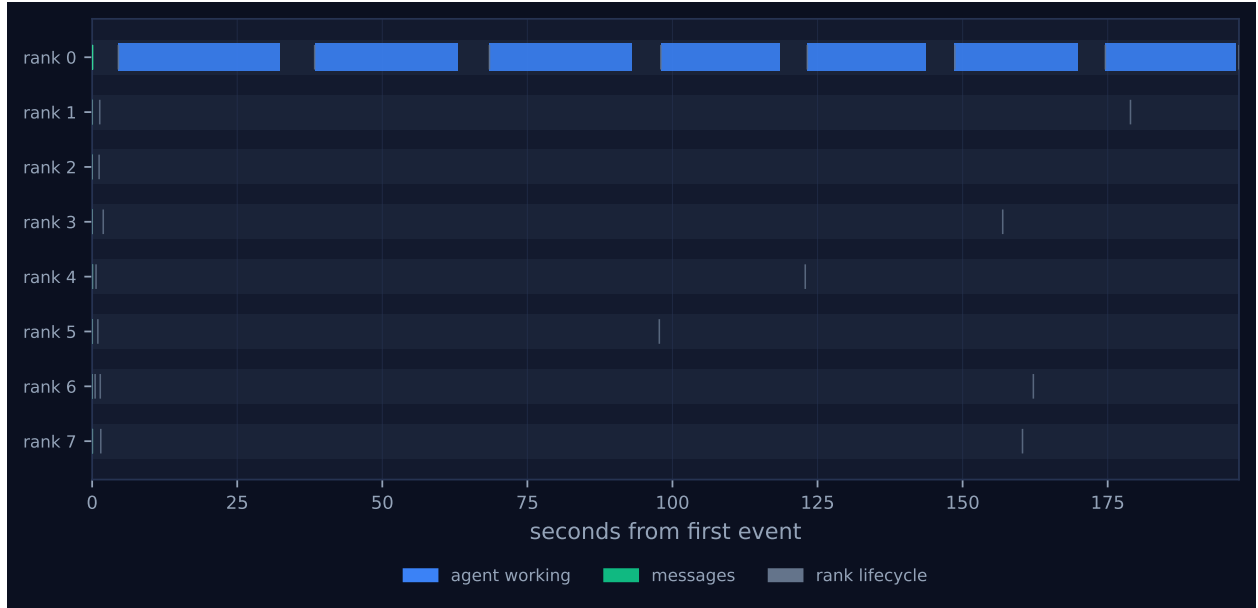


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer’s palette so the same shapes read the same way in both.

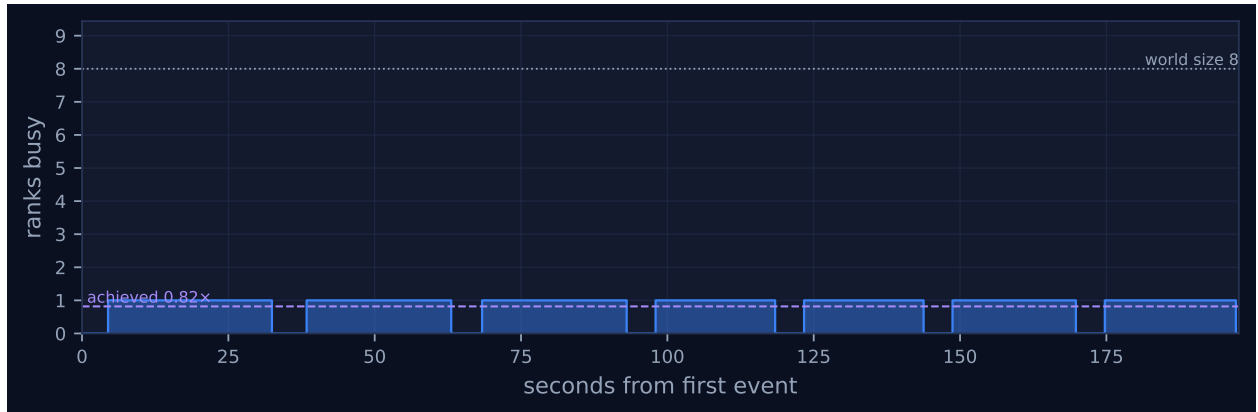


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

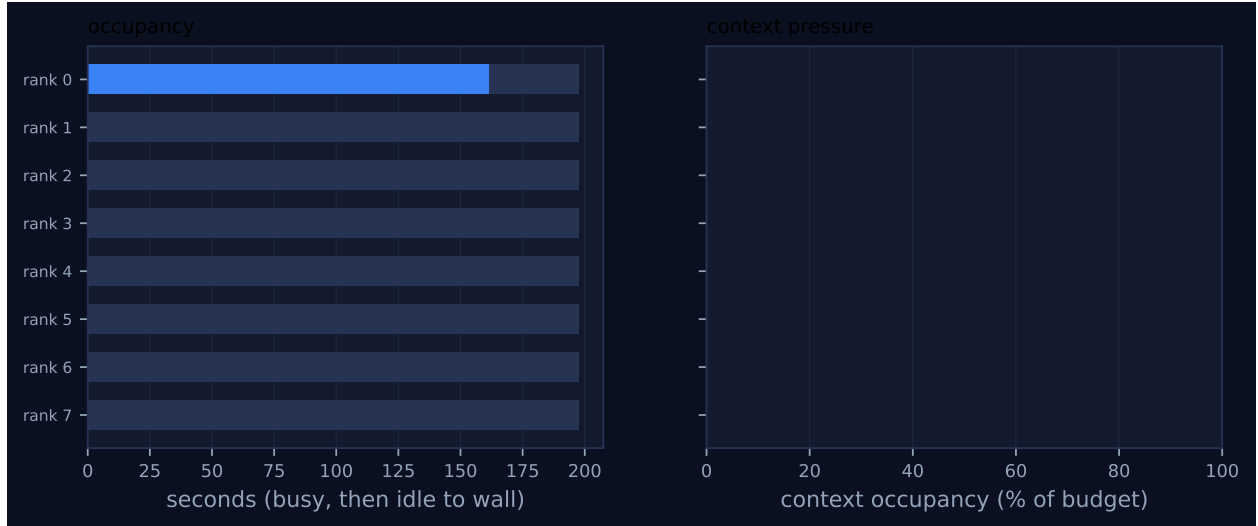


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	161.56	52.9	7	7	0	7	0	0.0	finalized
1	0.00	0.0	0	0	1	0	387	0.0	finalized
2	0.00	0.0	0	0	1	0	387	0.0	finalized
3	0.00	0.0	0	0	1	0	387	0.0	finalized
4	0.00	0.0	0	0	1	0	387	0.0	finalized
5	0.00	0.0	0	0	1	0	387	0.0	finalized
6	0.00	0.0	0	0	1	0	387	0.0	finalized
7	0.00	0.0	0	0	1	0	387	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	flat	–	8	8	1	1	7	7	2,709	197.62	197.61	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

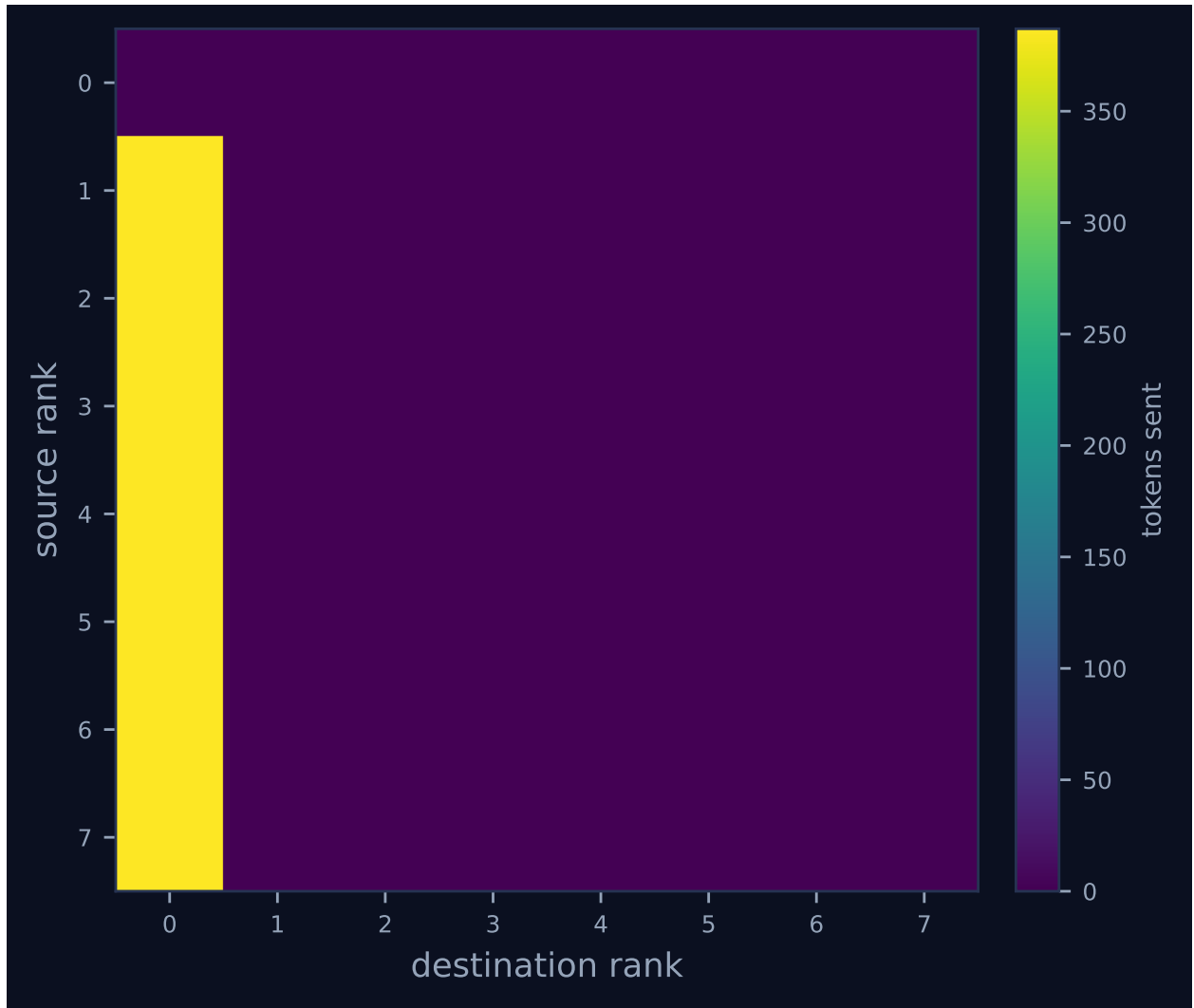


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

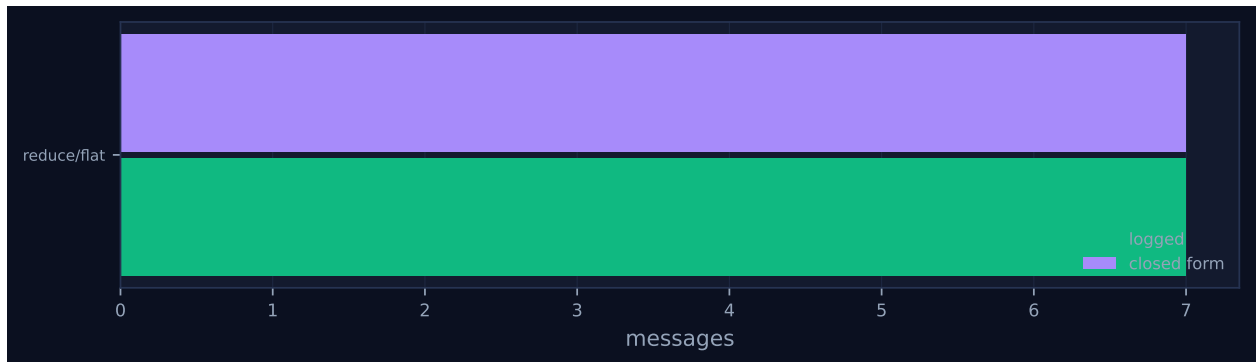


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

6 Reading the timeline

Figure 1 shows one occupied lane and seven empty ones. Rank 0 carries a train of seven contiguous bars across the whole 197.62s; ranks 1 to 7 carry two or three instantaneous ticks each inside the first 40ms and nothing after. That is the flat reduction drawn literally: every non-root rank sends its 387-token report to the root and returns, their `coll.reduce` records `fold_depth: 0`, and the root does all the work. Peak ranks busy is 1 of 8, serial fraction of active time 100%, load imbalance $8.00\times$ — the worst attainable at $p = 8$ and correctly reported — and rank 0’s 161.56s of busy time is the only non-zero entry in the per-rank table. The related `coordination_share` of 12.5% understates the waste for a different reason: the seven non-root ranks return from the reduce in milliseconds, so their 197s of idleness never enters `collective_rank_seconds`, where the chain’s blocked receives push the same quantity to 61.9%. This cell is far more wasteful of the pool than the chain and that particular metric says the reverse.

The bars are nearly equal in width while the work behind them is not, and that is worth stating because it contradicts the natural expectation. The seven merges took 27.79, 24.61, 24.51, 20.46, 20.42, 21.23 and 22.53s on prompts growing from 933 to 1,059 tokens and outputs growing from 354 to 698. There is no upward drift: the longest call is the first. The calibration block explains it, fitting a fixed cost of about 21s against a token-dependent term that only begins to matter above 2,788 tokens, an order of magnitude above anything here. The prompt stays near 900 tokens throughout even as the accumulator absorbs eight ranks, because the accumulator itself is being compressed as fast as it grows. Of the 197.6s, 161.6s is generation and 34.3s is waiting for a worker to claim a queued call, which I obtained by differencing every `broker.enqueue` → `broker.claim` pair in the log; so unlike the chain sibling this run’s wall time is mostly real model time. The 27 reattachments and the six events after `job.finish` at 197.62s — the log runs to 305s — are the pool recycling, and they are why the dashboard shows all eight ranks red and suspected `fail_stop` while `metrics.json` records `ok: true` and no failures.

The communication matrix, Figure 4, is a single column: seven edges into rank 0 at exactly 387 tokens each, 2,709 in total, and no accumulator ever crosses the fabric. That is the least traffic of the four cells — against the tree’s 3,004, the k -ary tree’s 2,913 and the chain’s 3,645 — and it is the trade the cost model predicts: one communication round bought by putting all seven applications on one rank’s critical path.

7 Interpretation

The structure is by construction and both checks pass: seven messages against $p - 1 = 7$ and one round against 1, so `model_checks_agree` is 1 of 1. The construction most in need of naming is the one the hypothesis usually gets wrong about this algorithm. A flat reduce is often described as folding all p inputs at the root in a single step, which would make it the shallowest arrangement available and the natural upper bound on retention. The implementation does no such thing: the `flat` branch of `reduce` gathers all p contributions, sorts them by source rank because the operator is declared non-commutative, and then applies the *binary* kernel $p - 1$ times in sequence, setting `fold_depth = p - 1 = 7` and `rounds = 1`. The trace confirms it directly — seven `agent.call` events on rank 0 labelled `merge:d1` through `merge:d7` — so this is a depth-7 cell sharing its depth with the chain, and the campaign contains three distinct depths (2, 3, 7) rather than four.

What the campaign varied, and whether it saturated, has to come out of the config and the store. Against `real-mb-fidelity` — same p , same executor, same algorithms — this campaign raised

facts from 4 to 12 and lowered **merge_budget** from 900 to 450, and added **kary** at **fanin**: 4. The root must render 96 items into half the space the baseline had for 32, and the baseline’s accumulators cost about 19.4 tokens per item, which puts the 450-token capacity near 23 items. It should have saturated four times over. It did not, and this cell shows both reasons in a single monotone sequence. Bytes per identifier fall 39.3, 29.0, 16.4, 15.6, 15.0, 14.6, 14.3 across the seven merges: the operator abandons prose for a packed table and, once it has, the marginal cost of admitting another rank settles at exactly 81 tokens — 454, 535, 617, 698 — or 6.8 tokens per item. Extrapolating that line backwards gives roughly $49 + 6.8n$ tokens for n items, so the 450-token budget is first crossed at about 59 items, and indeed the fourth merge is the first to exceed it at 454 tokens for 60. The remaining three overrun by 19%, 37% and 55%. Every one of them records **attempt**: 1. The contract in the fabric reads `{"name": "MergedReport", ..., "max_tokens": null, ..., "semantics": ""}` against **meta** of `{"max_tokens": 450}`: the budget reached the executor as a generation hint a worker may ignore, and never reached the checker that would have rejected it. Under a null bound, “zero contract violations” is uninformative rather than reassuring, and **make_tables.py** is right to classify this block empirically as *not capacity-bound*.

The comparison that makes this cell valuable is with its own twin in the corrected campaign. **inc-mb-fidelity-inc__fidelity-flat-f-k4-inc** is the same algorithm, the same $p = 8$, the same twelve items per rank and the same nominal 450 tokens, differing in exactly two respects: the contract carries **max_tokens**: 450 with a non-empty semantics clause, and the payload is high-entropy serials that cannot be paraphrased shorter. Its fold saturates at the same place — 437 tokens after absorbing rank 3, where this one reaches 454 after absorbing rank 4 — and then the two diverge completely. This run overruns and keeps everything; that one cannot overrun, has nothing left to compress, and so returns the byte-identical accumulator for **merge:d4** through **merge:d7**, erasing ranks 4 to 7 whole while every call still satisfies its contract. Retention 1.000 against 0.385, from one knob. That is the sharpest available demonstration that what governs retention is the enforced output budget rather than the shape of the fold tree, and it is why the depth-7 flat fold, the depth-3 tree and the depth-2 k -ary tree score identically within each campaign.

On cost this cell is the campaign’s second-worst and has no compensating advantage. Seven applications, all serial at the root: 197.62s against the binomial tree’s 77.56s and the k -ary tree’s 53.07s, at \$0.0709 against \$0.0625 and \$0.0369, on 6,353 prompt tokens. It is faster than the chain’s 1,376.98s, but that comparison is worthless because 1,100s of the chain is a worker failing to appear. Of the flagged findings, the serial-execution warning is the algorithm’s definition rather than a defect and is worth keeping for a reader meeting this pattern in a real harness; the $8.00\times$ imbalance is honest; the 27 reattachments are the broker’s normal mode; and the model check is the check passing.

8 Threats to this reading

The executor is real (**broker** $\times$ 8, **worker** $\times$ 27), so this is agent behaviour rather than a surrogate implementing a loss model, as the **fid-synth** and **fid-inc-smoke** cells do. The most serious threat is that retention 1.000 does not measure information transport. **make_fact_report** builds item i of rank r with throughput $100 + 7r + i$ under workload `['nominal', 'degraded', 'saturated'][i % 3]`, so the payload is a deterministic function of the identifier, and the packed encoding reproduces the rule literally — `[F-0-0]100n [F-0-1]101d ... [F-0-11]111s`, then `[F-1-0]107n ....` This cell is the one where that matters most, because its accumulator is a strictly increasing rank prefix in a visibly arithmetic sequence: after four ranks the root has seen 100–111, 107–118, 114–125 and

121–132, and the next rank’s twelve values are trivially predictable. A root that had lost rank 6’s message entirely could emit F-6-0 through F-6-11 with correct values and score as having retained them. Every retention claim here is a claim about identifier survival on a guessable payload; the `-inc` variant exists precisely because of this, and there the payload is scored as well as the label.

Second, two of this run’s seven results are byte-identical to results in sibling cells: 725368554341 (373 tokens, ranks 0–3) is also the binomial cell’s `merge:d2` and the k -ary cell’s `merge:k4:d1`, and e411144037d6 (698 tokens, all 96 items) is also the binomial cell’s `merge:d3` and the k -ary cell’s `merge:k2:d2`. The four cells ran sequentially against one persistent pool of Cursor subagents, so reuse was mechanically possible, and here three structurally different prompts — a fold of an accumulator with a leaf, a fold of two subtree accumulators, and a four-way variadic fold — produced the same object. I ran the determinism test the contract requires and it comes out against determinism: prompt digest `edf28c1217be` is shared between this run’s `merge:d1` and the binomial cell’s, and the results differ sharply (354 tokens against 211, different digests). Identical input does not give identical output here, so determinism cannot explain the identities. Convergence on the canonical packed encoding can, and is consistent with the identities occurring only among the dense late accumulators; cross-run reuse cannot be excluded from the trace.

Third, the 81-tokens-per-rank marginal cost is fitted to four points, from 60 to 96 items, after the operator has settled into the packed encoding; the first three merges do not lie on that line and the third is 52 tokens *below* the second despite carrying twelve more items. The sibling cell’s four structurally identical depth-1 merges returned 211, 269, 356 and 491 tokens on the same prompt structure, a $2.3\times$ spread, so per-item cost in the compressible regime is a choice the operator makes afresh and not a constant. The crossing point of 59 items is therefore accurate for this run and should not be read as a capacity for the configuration.

Finally, the usual accounting caveats. Every `msg.recv` records `admitted: false` because the collective’s internal receives pass `admit=False`, so context occupancy reads 0% even on a root that consumed 6,353 prompt tokens; this run says nothing about admission pressure, which is a real gap given that a flat reduction is exactly the pattern whose root should exhaust its context first. The \$0.0709 is modelled from \$3/Mtok in and \$15/Mtok out with `tokenizer_exact: false`, not billed. And `rank_position_correlation: 0.000` is not a fairness measurement but an undefined quantity, since at retention 1.000 the per-rank variance is zero and the code returns 0.0 by a guard.